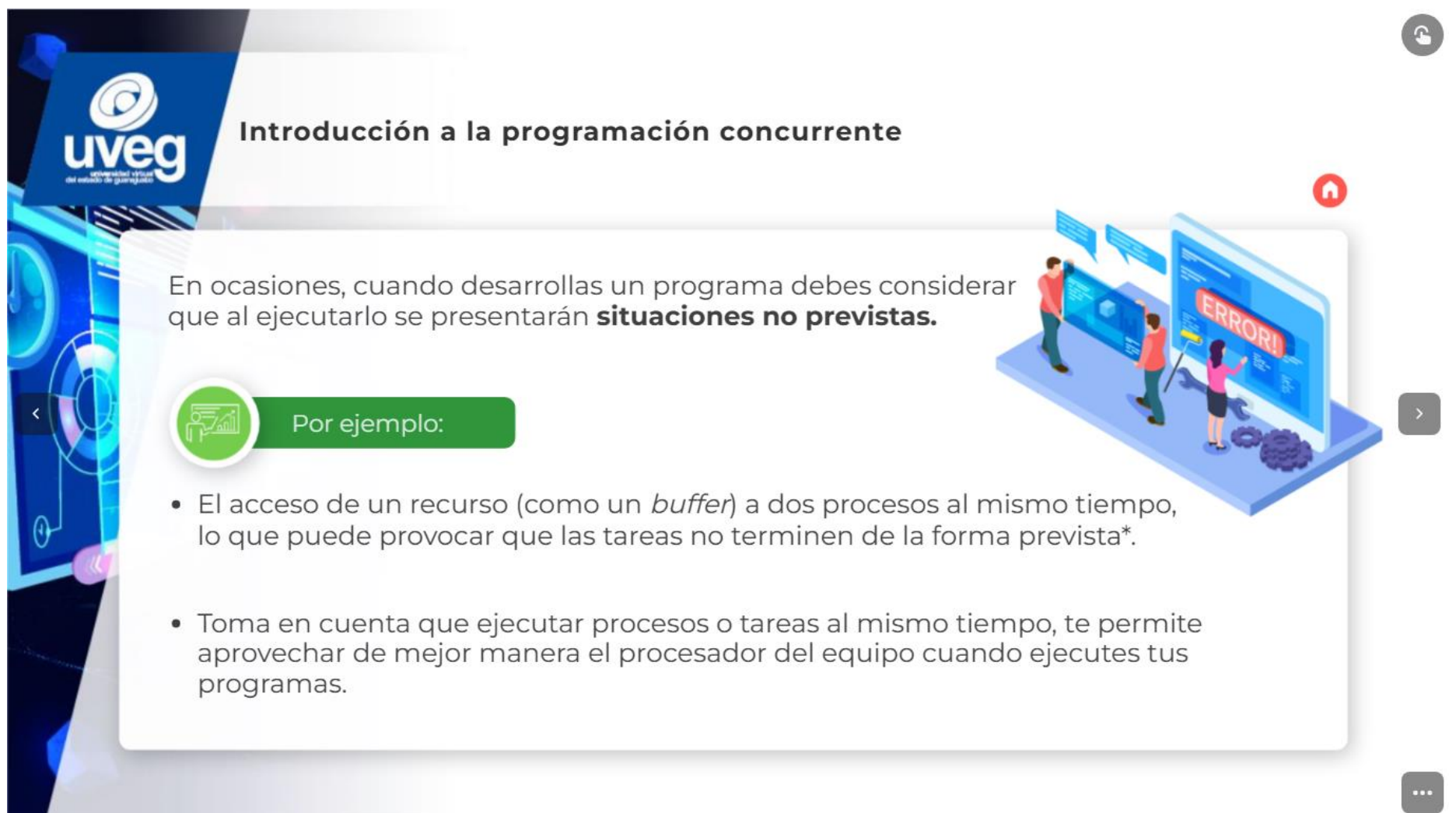




The banner features a futuristic, blue-toned digital interface on the left with various icons like a clock, a pie chart, and data streams. On the right, the text 'Introducción a la programación concurrente' is displayed in a bold, sans-serif font. Below the title is a dark button with the text 'INICIAR >'. The UVEG logo is positioned in the center-left of the banner.

Introducción a la programación concurrente

INICIAR >



This slide contains the main content of the introduction. It starts with the UVEG logo and the title 'Introducción a la programación concurrente'. Below this is a paragraph explaining that unexpected situations may arise when running a program. A green callout box with a person icon and the text 'Por ejemplo:' lists two bullet points. To the right, an illustration shows three people interacting with a large screen that displays a red 'ERROR!' message. Navigation icons are visible on the left and right sides of the slide.

Introducción a la programación concurrente

En ocasiones, cuando desarrollas un programa debes considerar que al ejecutarlo se presentarán **situaciones no previstas**.

Por ejemplo:

- El acceso de un recurso (como un *buffer*) a dos procesos al mismo tiempo, lo que puede provocar que las tareas no terminen de la forma prevista*.
- Toma en cuenta que ejecutar procesos o tareas al mismo tiempo, te permite aprovechar de mejor manera el procesador del equipo cuando ejecutes tus programas.



Introducción a la programación concurrente

¿Qué es la programación concurrente?

Es un **conjunto de técnicas** enfocadas en **solucionar problemas de ejecución** cuando se implementan al mismo tiempo **dos o más tareas**, considerando las posibles soluciones de comunicación entre ellas.

La concurrencia en la programación

- Permite un mejor aprovechamiento del *hardware* con el que se trabaja.





Introducción a la programación concurrente

¿Qué es la programación concurrente?

Observa el ejemplo y reafirma qué es la programación concurrente:

Piensa en un navegador

Al hacer una búsqueda, el navegador te permite:

- Ver varias imágenes
- Ver páginas *web*
- Previsualizar videos, entre otras cosas

Estas tareas son ejecutadas por diferentes **hilos**, los cuales **procesan múltiples tareas** al mismo tiempo dentro de la aplicación.





Introducción a la programación concurrente



Son **programas** a los que se les asigna un estado cuando están en ejecución, ya sea: creado, ejecutable, en ejecución, bloqueado o finalizado.

Todos ellos, independientes unos de otros.

Estos pueden ejecutarse al mismo tiempo, y cuando menos, dos de ellos comparten una relación (son parte del mismo trabajo u ocupan los mismos recursos), sucede lo que se conoce como **conurrencia de procesos**.



Introducción a la programación concurrente



Hasta ahora, solo has trabajado con un proceso en la **programación orientada a objetos (POO)**; sin embargo, la computación actual emplea en los sistemas operativos un concepto llamado **multiprogramación**.

La multiprogramación

Permite trabajar con más de un hilo (proceso) para incrementar el rendimiento de los programas al **paralelizar** (ejecutar más de un proceso al mismo tiempo) el código en ejecución.



Recuerda

La cantidad de **procesos** que se pueden **ejecutar** al mismo tiempo está limitada por el **número de núcleos** que contiene la computadora que ejecutará la aplicación.



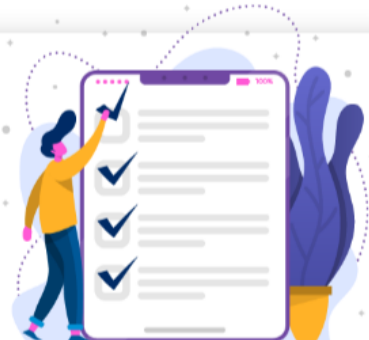


Introducción a la programación concurrente

Estados de un proceso

Cada proceso tiene asignado uno de los siguientes estados:

- **Creado.** El proceso ya fue **creado**, sin embargo, aún no se encuentra listo para ejecutarse.
- **Ejecutable.** El proceso está listo para ser **ejecutado**, pero se encuentra en espera de que haya un procesador disponible.
- **En ejecución.** El proceso está siendo **ejecutado**; esto sucede cuando ya fue asignado a un núcleo.
- **Bloqueado.** El proceso está en espera de que suceda alguna acción o el hilo ha dejado el procesador **libre** de forma voluntaria.
- **Finalizado.** El proceso ha **terminado** de realizar su tarea.

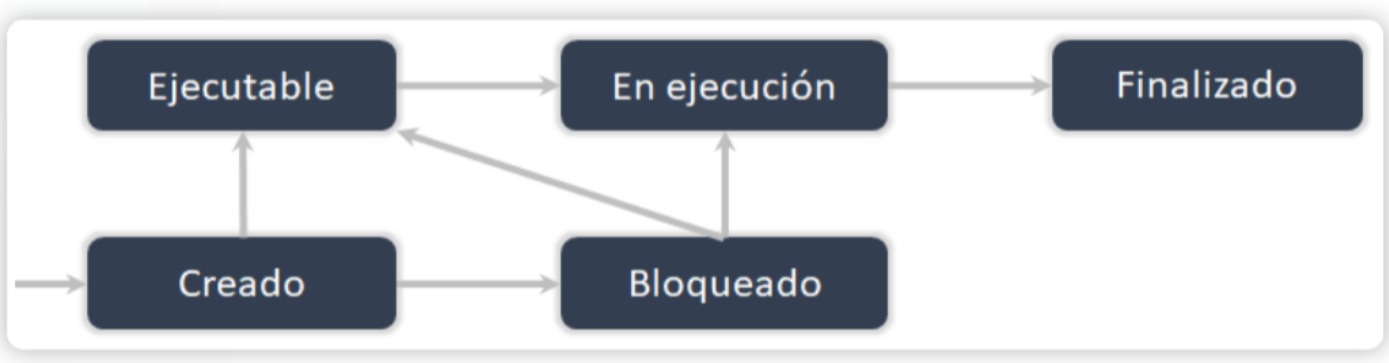




Introducción a la programación concurrente

Estados de un proceso

Observa esta imagen que ejemplifica los estados de un proceso



```
graph LR; Creado --> Ejecutable; Creado --> Bloqueado; Bloqueado --> En_ejecucion[En ejecución]; Ejecutable --> En_ejecucion; En_ejecucion --> Finalizado
```



Introducción a la programación concurrente

Colas de procesos

Existen varios mecanismos de sincronización; aunque, aquí solo veremos tres:

Semáforo



Paso de mensajes



Monitores





Introducción a la programación concurrente

Colas de procesos Semáforos

Para solucionar el problema

En 1965 el científico en programación Dijkstra (Tanenbaum, 2011, p. 128)

- Introdujo un tipo de variable llamada **semáforo**
- Cuyo valor sirve para almacenar las señales que despiertan al proceso productor.



El semáforo puede ser representado con:

- Un 0 para indicar que no hay señales guardadas.
- O con un número positivo para indicar si quedan señales pendientes.

Se propusieron dos operaciones para acceder al semáforo:
down y **up** (también conocidas como *sleep* y *wake up*).

Down

- Es una operación que verifica que el semáforo es mayor que 0; si es así, disminuye el valor (usando la señal de despertar).
- Si el valor es 0, pone en pausa la operación que se está ejecutando, sin terminar el *down*.

Up

- Es una operación que se encarga de aumentar el valor del semáforo para terminar las operaciones que *down* no ha concluido.
- Al terminar, el semáforo seguirá en 0 aunque tendrá que ejecutar un proceso menos.



Observa el ejemplo sobre las operaciones básicas con procesos **semáforos**:

```
public class Semaforo {
    private int valor;
    public Semaforo(int v) {
        valor = v;
    }
    public synchronized void down(){
        while(valor <= 0){
            try{
                wait();
            }catch(InterruptedException e){
            }
        }
        valor--;
    }
    public synchronized void up(){
        valor++;
        notify();
    }
}
```

```
public class Semaforo {  
    private int valor;  
    public Semaforo(int v) {  
        valor = v;  
    }  
    public synchronized void down(){  
        while(valor <= 0){  
            try{  
                wait();  
            }catch(InterruptedException e){  
            }  
        }  
        valor--;  
    }  
    public synchronized void up(){  
        valor++;  
        notify();  
    }  
}
```



Introducción a la programación concurrente

Colas de procesos
Paso de mensajes

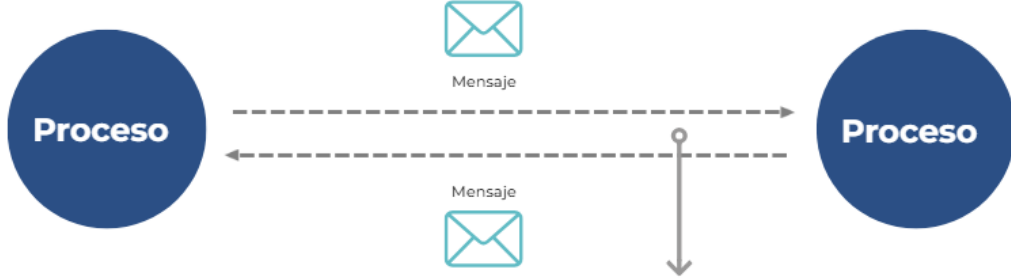


+

Ver más



Observa el ejemplo de **procesos** en el **paso de mensajes**:



```
graph LR; P1((Proceso)) -.->|Mensaje| P2((Proceso)); P2 -.->|Mensaje| P1; subgraph Canal [Canal de comunicación]; direction TB; C1[ ]; C2[ ]; end; C1 --- P1; C2 --- P2;
```





Introducción a la programación concurrente

Colas de procesos
Paso de mensajes

Observa el ejemplo de **procesos** en el **paso de mensajes**:

Es un **tipo de sincronización entre procesos** que a diferencia de los semáforos, no usa variables compartidas para comunicarse.



Ver más



Introducción a la programación concurrente

Colas de procesos
Paso de mensajes

Nota:

- La técnica empleada en los **semáforos** suele ser usada únicamente en **sistemas centralizados**

Donde el programador tiene la tarea de establecer los métodos de sincronización y comunicación.

Mientras que **el paso de mensajes** puede ser aplicado:

- en **sistemas distribuidos**
- mientras que el sistema operativo se encarga de facilitar los procesos de sincronización.

También se necesita un **canal de comunicación**, por el cual se garantice la entrega de los mensajes.

Para usar el **paso de mensajes** es necesario aplicar dos operaciones atómicas:

Send:
que permite enviar información a los procesos.

Receive:
que permite recibir información de otro proceso.



Introducción a la programación concurrente

Colas de procesos
Paso de mensajes



Existen diferentes tipos de comunicación entre los **procesos aplicados en el paso de mensajes**

Los siguientes son dos de los principales:

Comunicación síncrona

Es aquella que utiliza la participación simultánea entre el proceso **emisor** y el proceso **receptor**. En este tipo de comunicación el proceso emisor y el receptor del mensaje se conocen.

Comunicación asíncrona

Se basa en el uso de **buzones**, que son procesos de introducción y retiro de mensajes. Con el uso de los buzones, el emisor puede enviar los mensajes para que el receptor los recoja después.



Introducción a la programación concurrente

Operaciones básicas con procesos
Paso de mensajes



Observa el ejemplo del código mostrado para visualizar el funcionamiento de **DatagramSocket** y que el **DataPacket** no se encuentra completo:

Pseudocódigo de emisor

```
DatagramSocket socket = new DatagramSocket();  
String mensaje = "nuevo mensaje";  
byte[] datos = mensaje.getBytes();  
DatagramPacket paquete = new DatagramPacket(datos, datos.length, HostReceptor, 2345);  
socket.send(paquete);
```

Pseudocódigo de receptor

```
DatagramSocket socket = new DatagramSocket(2345);  
DatagramPacket dp = new DatagramPacket(buffer, MAXLEN);  
socket.receive(dp);  
len = dp.getLength();
```





Introducción a la programación concurrente

Operaciones básicas de procesos
Paso de mensajes



Pulsa para observar el ejemplo de los **procesos** con el **paso de mensajes**:

En la creación de **pasos de mensajes**, Java tiene clases especiales para implementar **sockets** que establezcan la conexión entre programas independientes.

Por ejemplo:

- `DatagramPacket`
- `DatagramSocket`

En un emisor

`DatagramPacket`: encapsula el mensaje y datos de envío.
`DatagramSocket`: lo envía mediante el método `send()`.

En un receptor

`DatagramPacket`: desencapsula el mensaje.
`DatagramSocket`: lo recibe mediante el método `receive()`.



Introducción a la programación concurrente

Colas de procesos
Monitores



- Permiten que el programador defina operaciones exactas aplicables a un tipo de datos que se excluyen entre sí, estos datos suelen ser **abstractos**.
- Trabajan bajo los principios de la **encapsulación**, por lo cual, el acceso a esta información solo se puede dar dentro del propio monitor.
- Son considerados mecanismos de programación de mayor nivel que se encargan de proteger las secciones críticas, con más de un hilo ejecutándose en ellas.





Introducción a la programación concurrente

Colas de procesos
Monitores



- El **uso de semáforos** es una buena forma de sincronización entre procesos.
- Aunque siempre está latente el riesgo de que los programadores cometan errores, como el **intercambio de un *wait* por un *signal***, lo cual puede ser crítico.

Para **evitar este tipo de errores**, se han elaborado diferentes técnicas; entre ellas, destaca el **uso de monitores**.



Nota:

Esta manera de trabajar garantiza la ejecución de un proceso por monitor a la vez.
Lo que implica que el desarrollador no programa de forma explícita las restricciones para la sincronización de procesos.



Introducción a la programación concurrente

Operaciones básicas con procesos
Monitores



A continuación, observa el ejemplo de un monitor que se utiliza para evitar la modificación de una variable (tipo de animal) por otro hilo.

Esto garantiza la exclusión mutua

```
class Animal {  
    private String tipoAnimal;  
    public Animal (String tipo) {  
        tipoAnimal = tipo;  
    }  
    public synchronized void setTipo (String tipo) {  
        tipoAnimal = tipo;  
    }  
    public synchronized String getTipo () {  
        return tipoAnimal;  
    }  
}
```





Introducción a la programación concurrente

Colas de procesos
Productor - Consumidor



Existe un problema en la interacción entre procesos denominado como **productor-consumidor**.

Éste sucede cuando:

- El **productor** (proceso 1) coloca información dentro del *buffer*, y
- El **consumidor** (proceso 2) extrae información del *buffer*, pero...

¿Dónde está el problema?

Imaginemos que ambos procesos están cumpliendo con sus funciones

Cuando el productor trata de colocar un nuevo elemento en el buffer, pero éste se encuentra lleno.

Una opción para solucionar el problema es:

- Mandar a dormir al productor, mientras el consumidor extrae del buffer uno o más elementos.
- El productor será despertado después para que continúe con su trabajo.

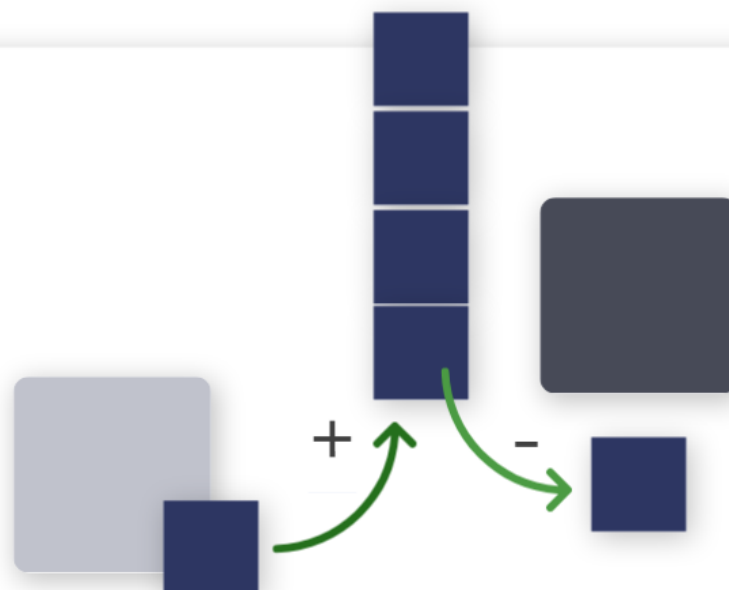


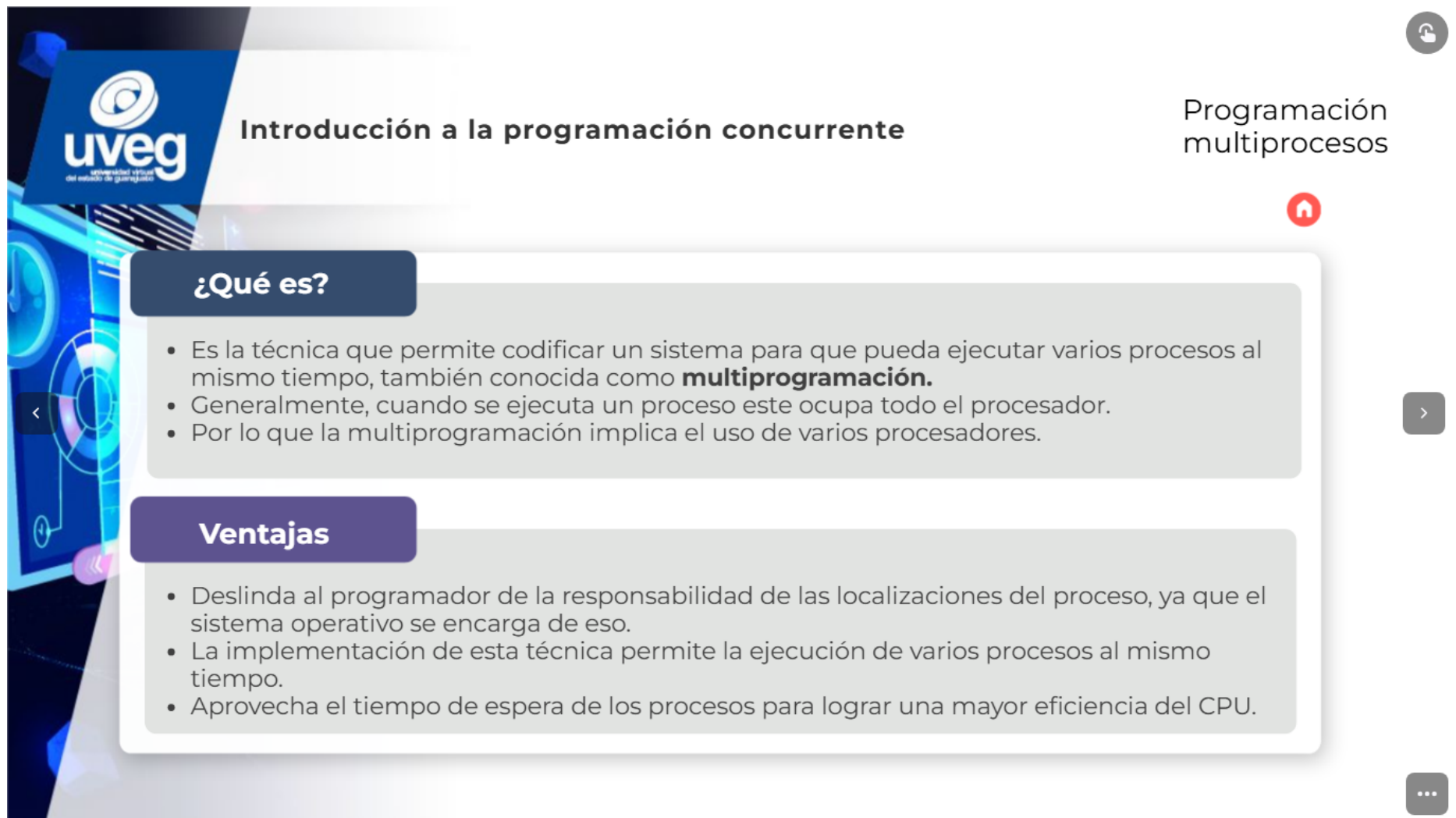
Introducción a la programación concurrente

Colas de procesos
Productor - Consumidor



Observa el ejemplo de productor-consumidor





Introducción a la programación concurrente

Programación multiprocesos

¿Qué es?

- Es la técnica que permite codificar un sistema para que pueda ejecutar varios procesos al mismo tiempo, también conocida como **multiprogramación**.
- Generalmente, cuando se ejecuta un proceso este ocupa todo el procesador.
- Por lo que la multiprogramación implica el uso de varios procesadores.

Ventajas

- Deslinda al programador de la responsabilidad de las localizaciones del proceso, ya que el sistema operativo se encarga de eso.
- La implementación de esta técnica permite la ejecución de varios procesos al mismo tiempo.
- Aprovecha el tiempo de espera de los procesos para lograr una mayor eficiencia del CPU.

¿Qué es?

- Es la técnica que permite codificar un sistema para que pueda ejecutar varios procesos al mismo tiempo, también conocida como **multiprogramación**.
- Generalmente, cuando se ejecuta un proceso este ocupa todo el procesador.
- Por lo que la multiprogramación implica el uso de varios procesadores.

Ventajas:

- Deslinda al programador de la responsabilidad de las localizaciones del proceso, ya que el sistema operativo se encarga de eso.
- La implementación de esta técnica permite la ejecución de varios procesos al mismo tiempo.
- Aprovecha el tiempo de espera de los procesos para lograr una mayor eficiencia del CPU.



Introducción a la programación concurrente

El uso de la programación concurrente

- Ha **aumentado su importancia** en los últimos años al grado de ser necesariamente **incluida en los programas**, lo que permite cumplir con las exigencias de cualquier usuario.
- **Los programas** deben ser capaces de ejecutar más de un proceso al mismo tiempo y de ofrecer un mejor aprovechamiento de los recursos disponibles en el equipo (procesadores).
- **Si nuestros proyectos no cumplen con ese rendimiento**, dejamos en manos de la competencia un nicho de oportunidades comerciales y laborales.



Material Apoyo: <https://sites.google.com/site/concurrente9/home?authuser=0>

Programación Concurrente y Paralela

- 1.Introducción
- 2.Algoritmo de Dekker y Peterson
- 3.Procesos e Hilos
- 4.Sincronización
- 5.Semáforos
- 6.Monitores
- 7.Lockes
- 8.Sockets
- 9.Paso de mensajes y Tubería
- MPI (Message Passing Interface)

1.Introducción

La programación paralela o programación concurrente es una técnica de programación basada en la ejecución simultánea, bien sea en un mismo ordenador (con uno o varios procesadores) o en un cluster de ordenadores, en cuyo caso se denomina computación distribuida.

Los sistemas multiprocesador o multicomputador consiguen un aumento del rendimiento si se utilizan estas técnicas. En los sistemas monoprocesador el beneficio en rendimiento no es tan evidente, ya que la CPU es compartida por múltiples procesos en el tiempo, lo que se denomina multiplexación.